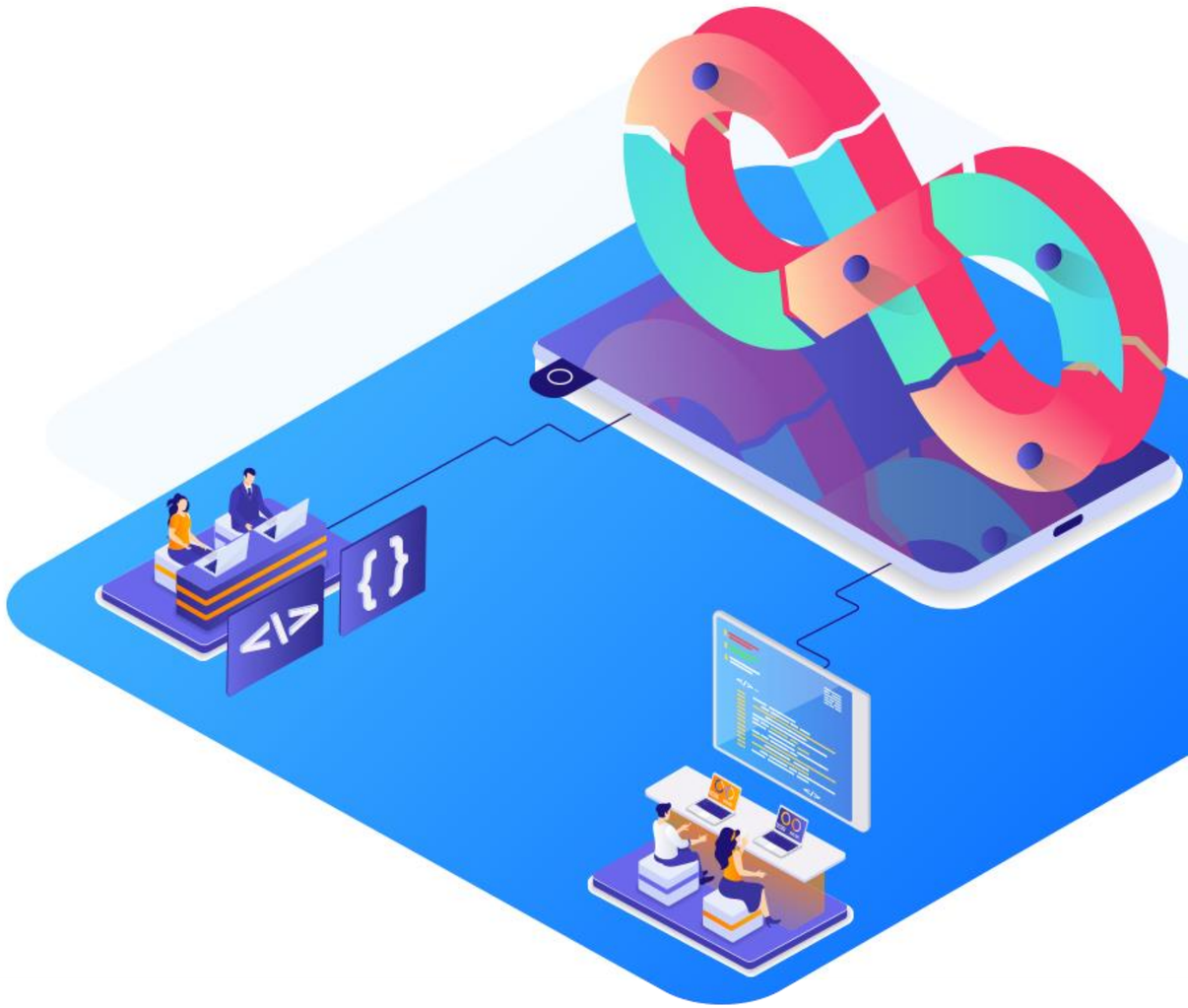
DevOps on AWS



Container Orchestration on AWS

Learning Objectives

By the end of this lesson, you will be able to:

- Use Docker to create and manage containers to isolate and run applications in a portable and secure environment
- Develop and deploy microservices with Docker for enhanced software scalability and reliability
- Set up Kubernetes on AWS EC2 for container orchestration across servers
- Employ EKS for cluster scaling, application deployment, and AWS service integration
- Apply best practices in EKS for performance, availability, and security optimization

Introduction to Containerization and Orchestration

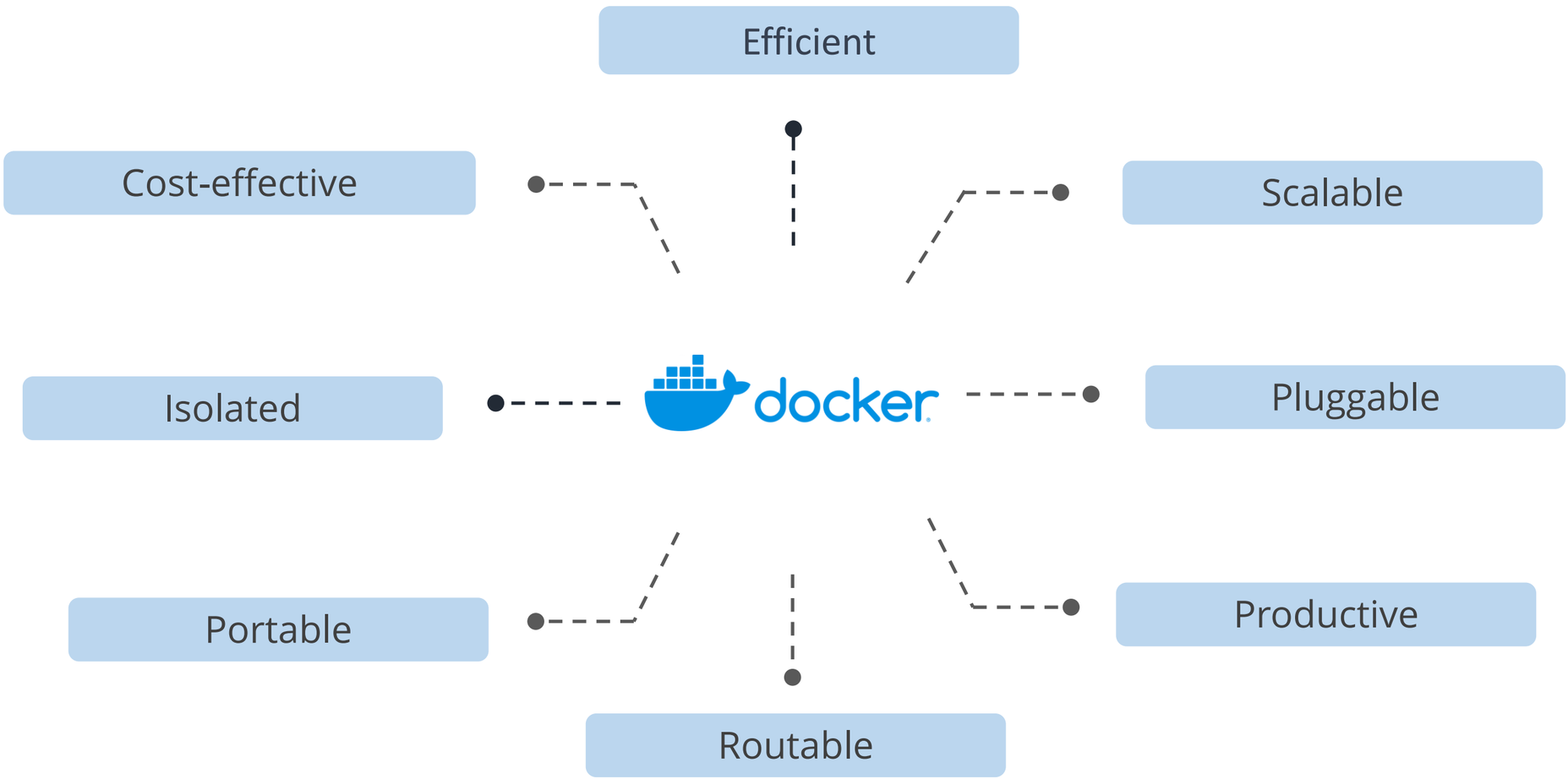
Docker Overview

Docker is the most popular and widely used container platform that enables developers and operators to build, run, and share applications using containers.

It packages software into containers, which include everything it needs to execute, including libraries, system tools, code, and runtime.

Benefits of Docker

Docker-based applications consume less memory, start and stop faster, and use system resources better. It is:



Docker Tools and Services

Docker provides a set of tools and services that simplify the creation and management of containers, such as:

Docker Engine: The core component that runs and manages containers on a host system

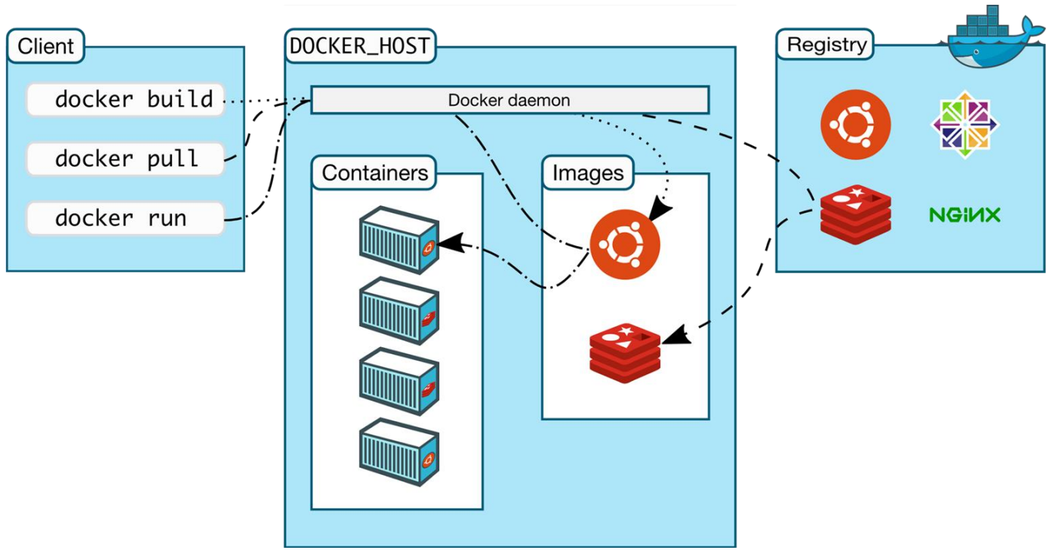
Docker Hub: A cloud-based registry service that stores and distributes container images

Docker Compose: A tool that defines and runs multi-container applications using a YAML file

Docker Swarm: A native clustering and orchestration solution for Docker containers

Docker Architecture

Docker is built on a client-server model, which includes the three main components: Docker Client, Host, and Registry.

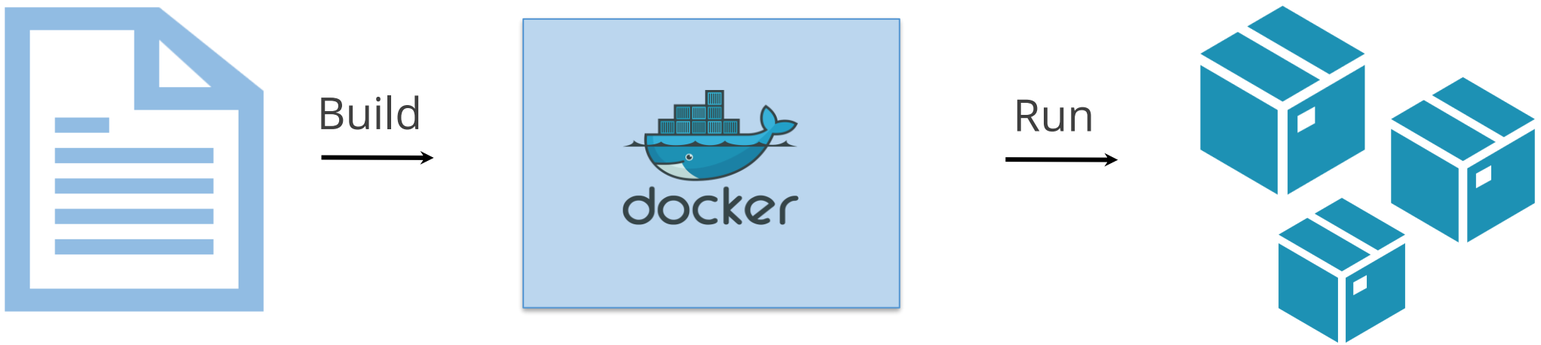


The Docker client communicates with the Docker daemon, which handles the construction, execution, and distribution of the Docker containers.

Source:: https://docs.docker.com/engine/images/architecture.svg

What Is a Docker Image?

It is a read-only template containing the instructions and files needed to create a container.



Dockerfile

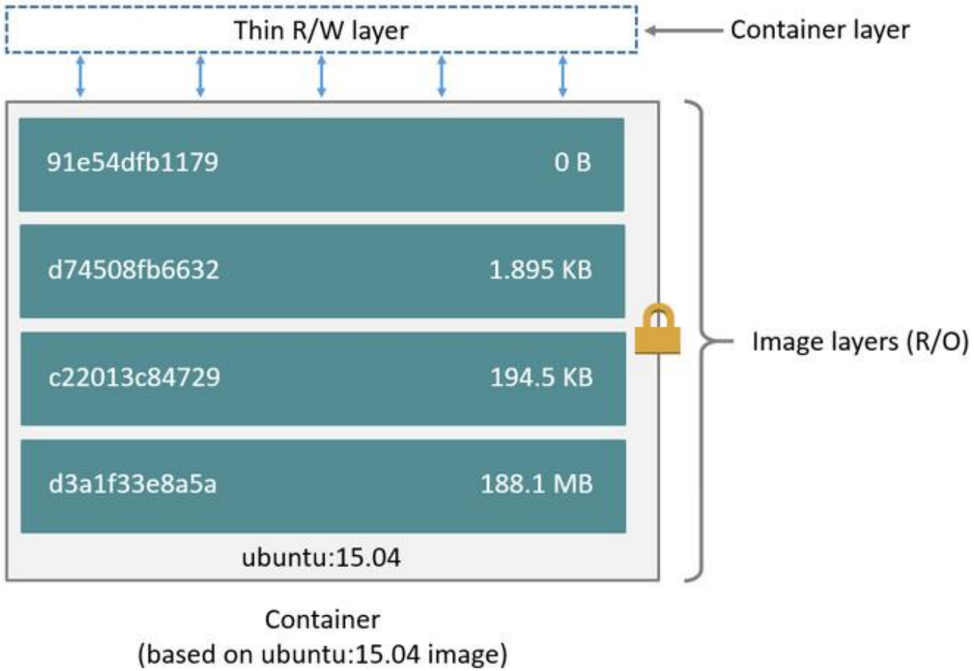
Docker image

Docker container

Multiple containers can be created from the same image, and each container will have its isolated environment.

Images and Layers

A Docker image consists of multiple layers stacked on top of each other.

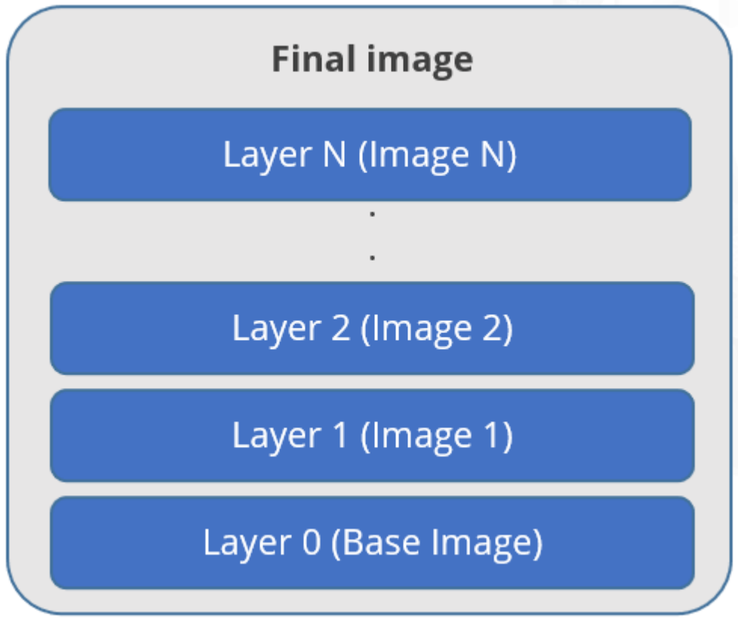


Images and Layers

Each layer represents a change or an addition to the image, such as installing a package, copying a file, or running a command.

Dockerfile

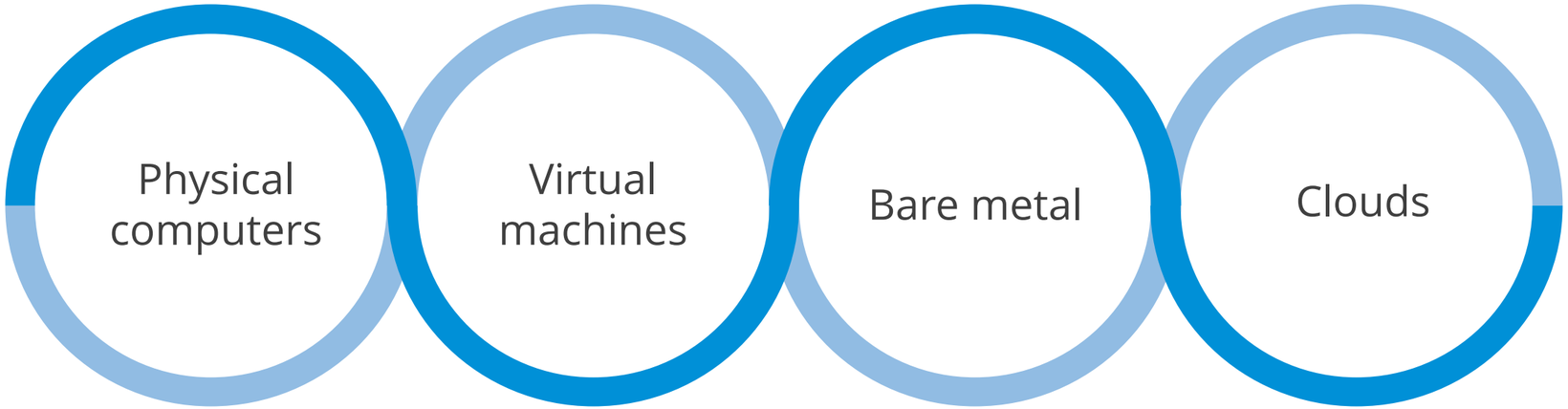
```
FROM ubuntu 20.04                #Layer 0
ENV APP nginx                    #Layer 1
RUN apt-get install -y APP       #Layer 3
WORKDIR /var/www/html/           #Layer 4
ADD index.html                   #Layer 5
EXPOSE 80                        #Layer 6
CMD [“nginx”,”daemon off”]       #Layer 7
```



What Is a Docker Container?

A Docker container is a running instance of a Docker image with its isolated filesystem, network, and processes.

Docker containers can run on:



Prepackaging allows the software to run fast and reliably from one computing environment to another.

Benefits of Containers

Containers consume fewer resources than virtual machines or other outdated program architectures, as they are:

Lightweight

Scalable

Portable

Agile

Problems Containers Solve

Containers are versatile and solve a broad range of IT problems throughout an application's life cycle.

- Ensure that software runs properly when moved between computing environments
- Increase efficiency by eliminating the need for a separate hypervisor
- Facilitate micro-service deployments
- Eliminate conflicts and dependencies between multiple applications

Assisted Practice

Deploying MySQL database as Docker container **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to deploy a MySQL database as a Docker container on an AWS EC2 instance with Ubuntu.

Outcome:

By performing this demo, you will learn how to deploy a MySQL database as a Docker container on an AWS EC2 instance with Ubuntu, gaining practical skills in cloud infrastructure, containerization, and database management. This experience will enhance your proficiency in setting up and managing Docker containers in a cloud environment, which is crucial for modern DevOps practices.

Note: Refer to the demo document for detailed steps:
01_Deploying_MySQL_Database_as_Docker_Container

Assisted Practice: Guidelines

Steps to be followed:

1. Set up an EC2 instance
2. Install the Docker on Ubuntu
3. Run the MySQL container
4. Connect to MySQL server within container

ASSISTED PRACTICE

Quick Check

Your team is preparing to deploy an application using Docker. You need a template that includes the application code, dependencies, and environment settings to consistently create containers in different environments. What should you use to achieve this?

A. To store application logs in a container

B. To create a network for containers

C. To provide a read-only template for creating containers

D. To manage container orchestration

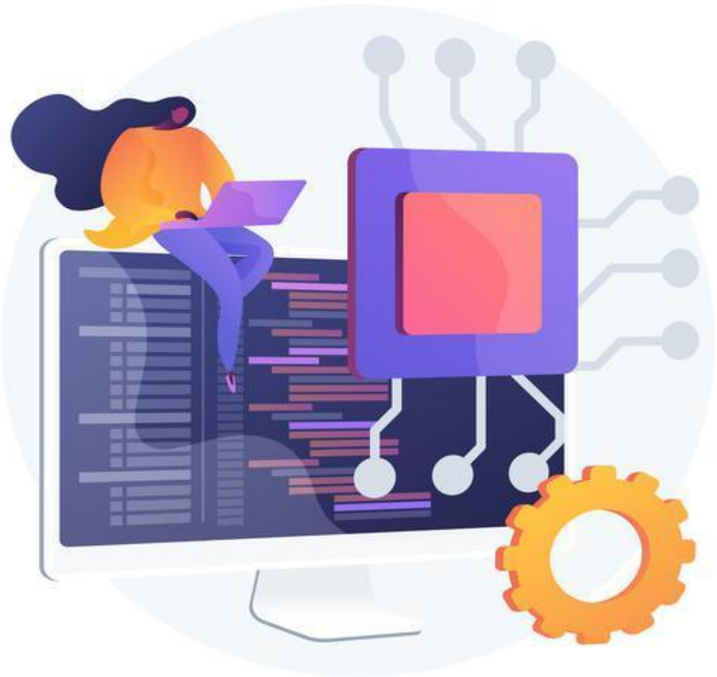
Kubernetes Fundamentals

What Is Kubernetes?

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

Kubernetes Design Principles

Kubernetes is a collection of building components. The components work together to offer mechanisms for deploying, maintaining, and scaling applications.

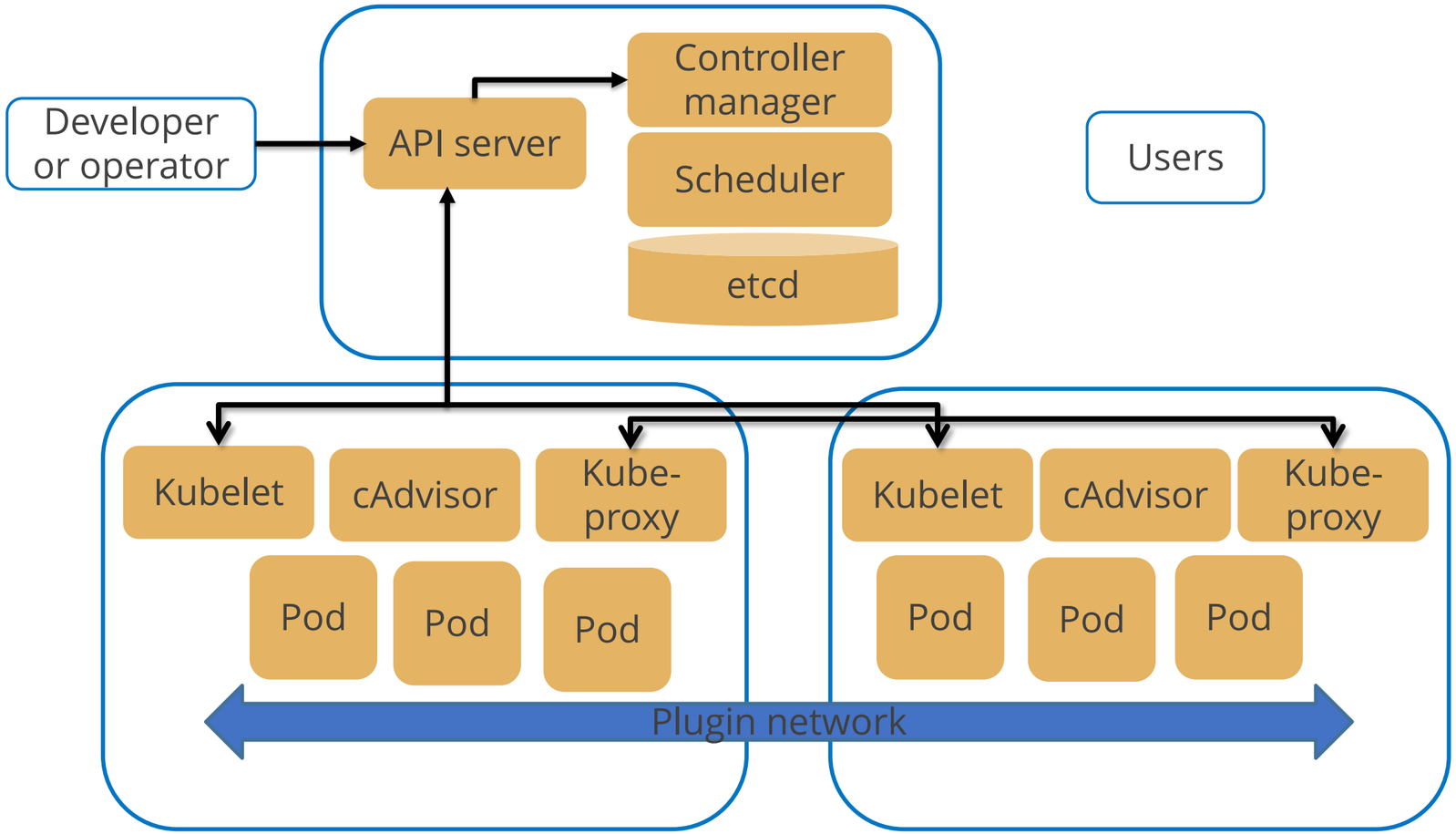


This platform gains control over computing and storage resources by defining resources.

Kubernetes is coupled and expanded to handle a variety of workloads and scheduling scenarios.

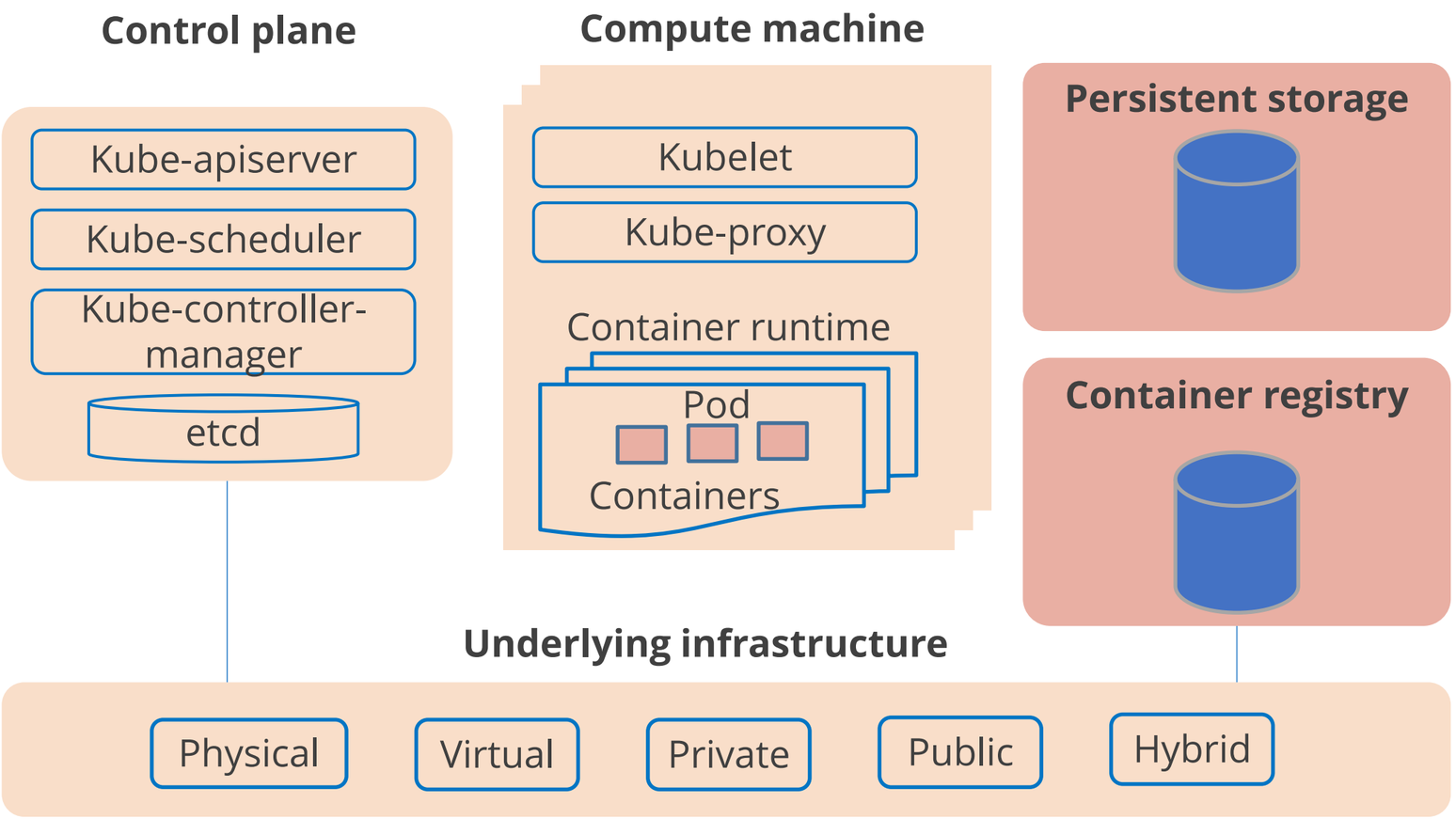
Kubernetes Architecture

A Kubernetes architecture consists of a master (control plane), a distributed storage system (etcd) for maintaining cluster state consistency, and several cluster nodes.



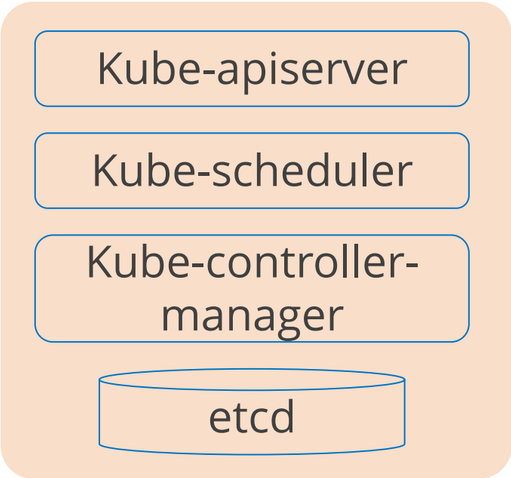
Kubernetes Cluster

A Kubernetes cluster is a group of physical or virtual machines called nodes to manage containerized apps and services.



Control Plane

The control plane (master) is in constant contact with the computing machines and ensures that the containers are running on the necessary resources.



Kube-apiserver handles internal and external requests.

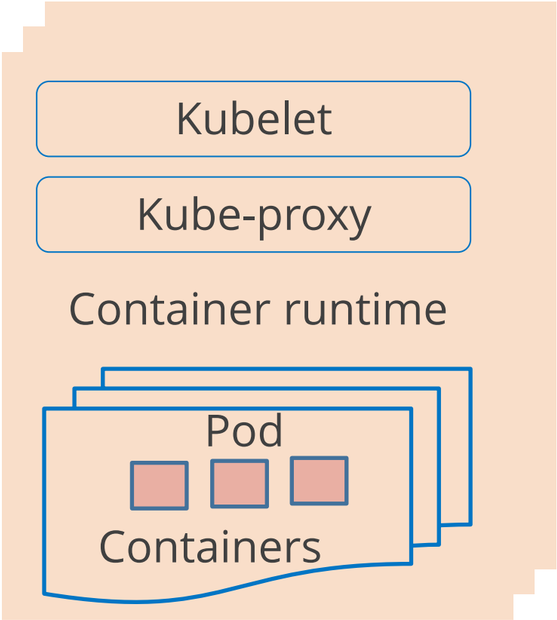
Kube-scheduler schedules the Pod to an appropriate node.

Kube-controller-manager helps run the cluster.

etcd stores configuration data and information about the state of cluster lives.

Compute Machine (Worker)

A Kubernetes cluster requires at least one compute node for scheduling pods. The control plane is not schedulable by default due to its default taints.



Kubelet communicates with the control plane.

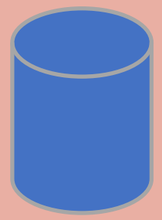
Kube-proxy is a network proxy that facilitates Kubernetes network services.

Container runtime manages and runs the components required to run containers.

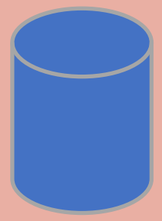
Pod represents a single instance of an application.

Storage and Registry

Persistent storage



Container registry



Persistent storage manages application data attached to a cluster.

Container registry stores the container images that the Kubernetes relies on.

Features of Kubernetes

Kubernetes offers the following features to its users:

- Storage
- Secure configuration management
- Autoscaling
- Automated rollouts and rollbacks
- Service discovery and load balancing

Understanding Pods

Pods are the smallest deployable computing units in Kubernetes. They can be created and managed within a Kubernetes cluster.

Pods in a Kubernetes Cluster are primarily used in two ways:

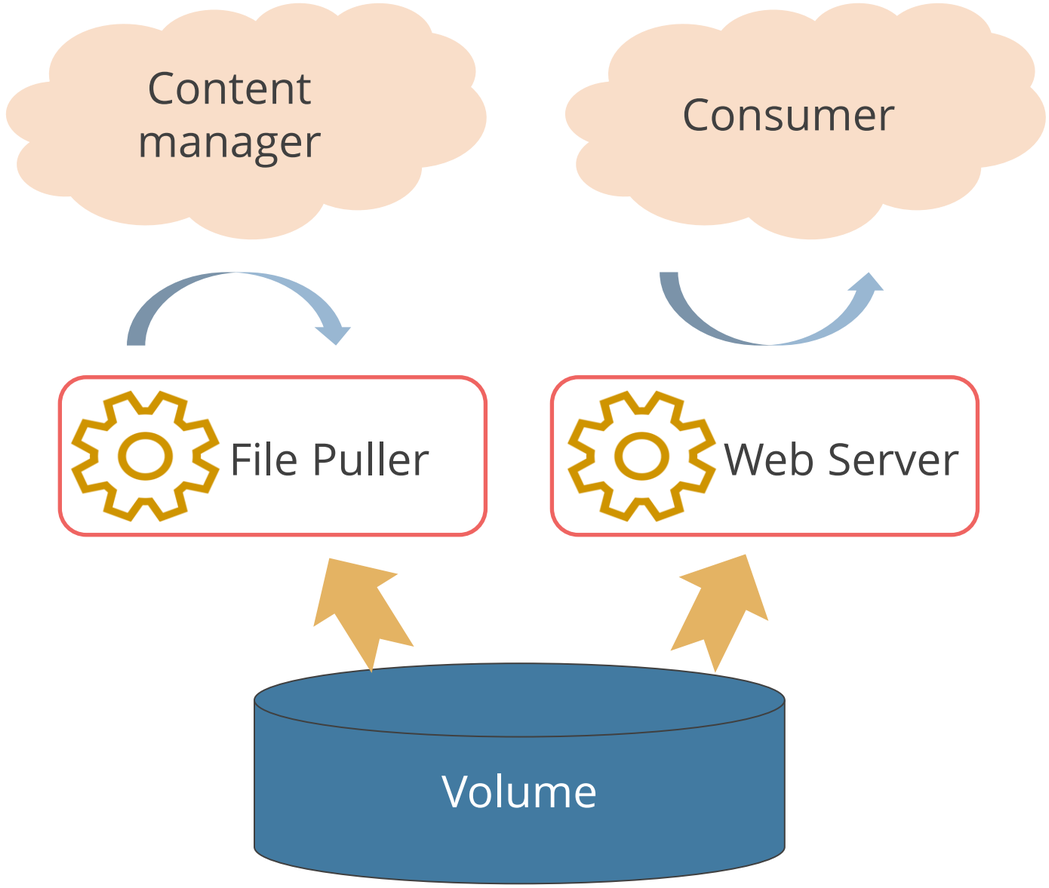
1. **Pods that run a single container**: The most common Kubernetes use case follows the **one-container-per-Pod** model.

2. **Pods that run multiple containers** should collaborate with each other.

©Simplilearn. All rights reserved.

How Pods Manage Multiple Containers?

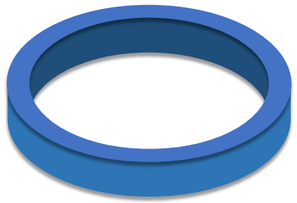
Pods are designed to support multiple cooperating processes (as containers) that form a cohesive unit of service.



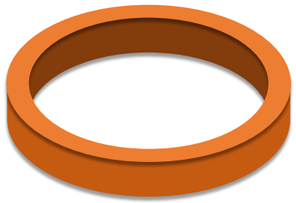
Pods and Controllers

Workload resources create and manage one or more pods.

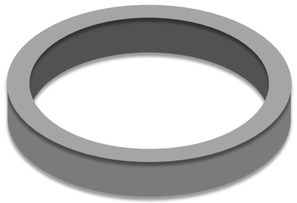
Examples of workload resources:



Deployment



StatefulSet



DaemonSet

Pod Template

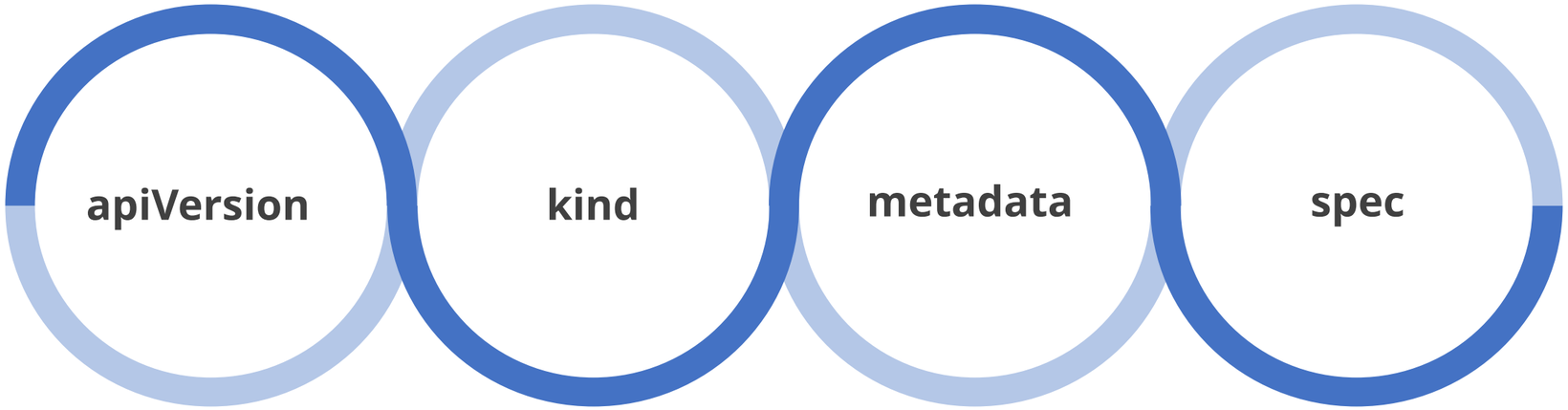
Pod templates are specifications for creating pods and are included in workload resources such as deployments, Jobs, and Daemon Sets.

Sample pod template:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

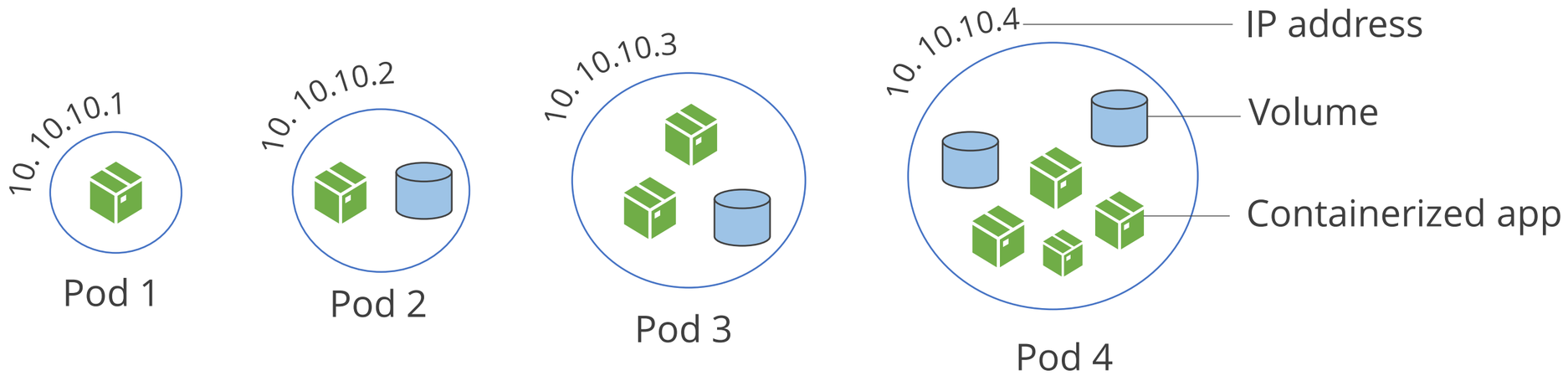
Required Fields in .yaml File

In the **.yaml** file, a set of values creates a Kubernetes object for the following fields:



Pod Update and Replacement

The controller does not update or patch existing pods when the pod template for a workload resource is updated or changed.



The controller creates new pods based on the updated template.

Types of Controllers

DaemonSet: Ensures all nodes run a copy of a Pod

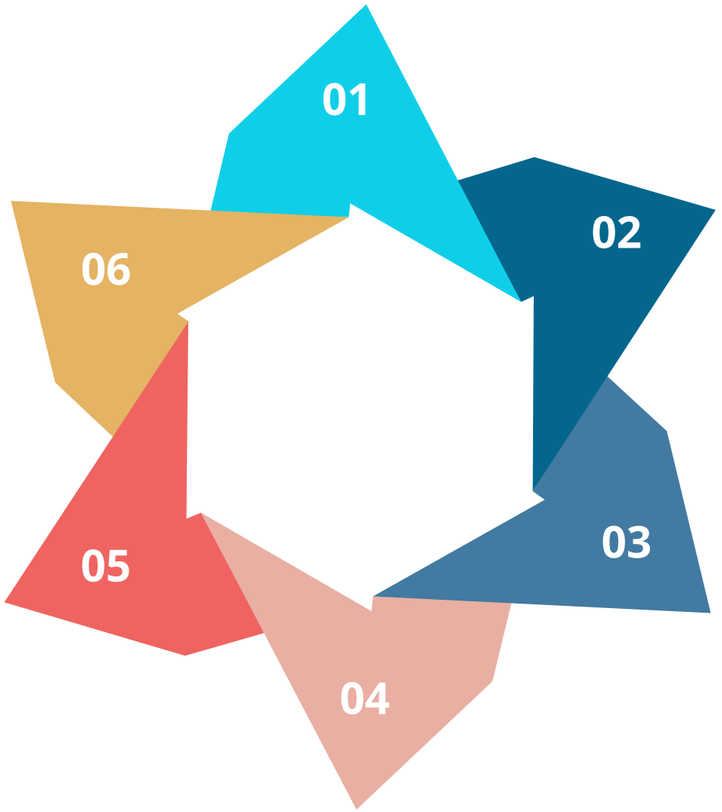
ReplicaSet: Creates a stable set of Pods

Deployment: Maintains ReplicaSets with a desired configuration

StatefulSet: Manages stateful applications

Job: Creates one or more short-lived Pods

CronJob: Creates a job on schedule



Assisted Practice

Setting up Kubernetes Cluster on EC2 Instances **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to set up a Kubernetes cluster on Amazon EC2 instances, ensuring seamless communication between a master node and a worker node.

Outcome:

By performing this demo, you will learn how to set up a Kubernetes cluster on Amazon EC2 instances, ensuring seamless communication between a master node and a worker node. This will enhance your skills in Kubernetes cluster management and cloud infrastructure setup, crucial for efficient container orchestration.

Note: Refer to the demo document for detailed steps:
02_Setting_up_Kubernetes_Cluster_on_EC2_Instances

Assisted Practice: Guidelines

Steps to be followed:

1. Set up the EC2 instances
2. Configure the Kubernetes master node
3. Connect the worker instance to the master node

ASSISTED PRACTICE

Quick Check

Your team is troubleshooting an issue where external tools and users are unable to interact with your Kubernetes cluster. You suspect the problem lies in the control plane component responsible for handling API requests. Which component should you focus on?

A. Kube-apiserver

B. Kube-scheduler

C. Kube-controller-manager

D. etcd

Amazon Elastic Kubernetes Service

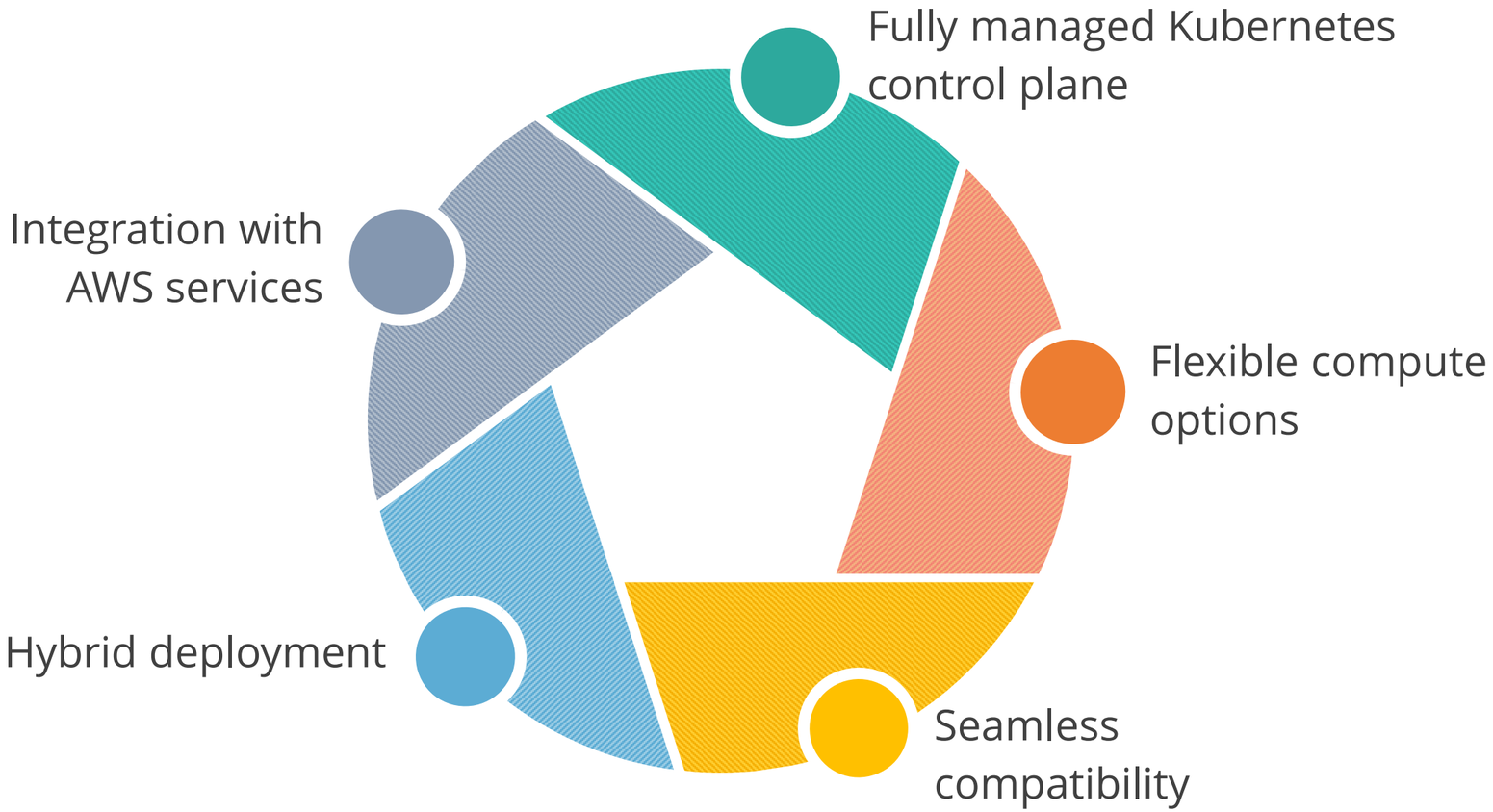
What Is Amazon EKS?

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed Kubernetes platform on AWS.

- It offers a reliable, secure, and scalable control plane across multiple Availability Zones.
- It integrates with AWS services like VPC, IAM, EBS, and ELB.
- EKS also supports running Kubernetes workloads on AWS Fargate, a serverless container compute engine that eliminates server provisioning and management.

Features of Amazon EKS

Amazon EKS offers the following features and benefits:



Requirements for Amazon EKS

To use Amazon EKS, user needs the following:

01 An AWS account and access to the AWS Management Console, AWS CLI, or AWS SDKs

02 A VPC with public and private subnets and security groups to allow communication between the control plane and the nodes

03 An IAM role for the EKS cluster and an IAM role for the node groups or Fargate profiles

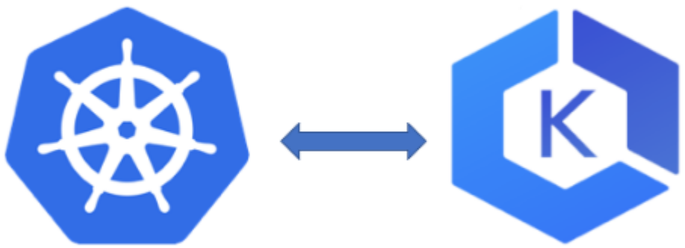
Requirements for Amazon EKS

To use Amazon EKS, user needs the following:

04 A Kubernetes tool such as kubectl, eksctl, or Helm to interact with the cluster and deploy applications

05 Alternatively, a container registry such as Amazon ECR or Docker Hub to store and pull the container images

AWS EKS vs. Self-Managed Kubernetes



- AWS EKS provides a fully managed Kubernetes service that simplifies the installation, operation, and maintenance of Kubernetes clusters on AWS.
- Self-managed Kubernetes requires setting up, configuring, and managing one's Kubernetes clusters on AWS or other platforms.

AWS EKS vs. Self-Managed Kubernetes

The following table compares some attributes of AWS EKS and self-managed Kubernetes:

Criteria	AWS EKS	Self-Managed Kubernetes
Control plane	AWS handles control plane updates and availability.	Users manage control plane installation and operation.
Nodes	Users can use Fargate, managed nodes, or self-managed nodes.	Users provision and manage nodes themselves.
Networking	AWS provides VPC CNI plugin and integration with ELB and Route 53.	Users choose and install a networking plugin and configure it with AWS services.

AWS EKS vs. Self-Managed Kubernetes

The following table compares some attributes of AWS EKS and self-managed Kubernetes:

Criteria	AWS EKS	Self-Managed Kubernetes
Security	AWS provides IAM integration and encryption for data.	Users set up the authentication and encryption tools.
Cost	AWS charges you for control plane, nodes or Fargate, and AWS services.	Users pay for nodes and AWS services and operational cost of control plane.

EKS Components: Control Plane

The control plane is the brain of the Kubernetes cluster that manages the state and configuration of the cluster.

The control plane consists of the following components:

In Amazon EKS, the control plane is fully managed by AWS and runs across multiple Availability Zones for high availability and fault tolerance.

EKS Components: Nodes

Nodes are worker machines that run the containerized applications in the Kubernetes cluster.

Nodes consist of the following components:

kubelet | kube-proxy | Container runtime

EKS Components: Nodes

In Amazon EKS, users have the following options to run nodes:

- **AWS Fargate:** A serverless compute engine for containers that automatically provisions and scales nodes for pods.
- **Managed node groups:** Groups of Amazon EC2 instances that are automatically registered to the cluster and managed by AWS.
- **Self-managed nodes:** Amazon EC2 instances that the users provision and manage themselves and manually register to the cluster.

EKS Components: Pods

Pods are the smallest and simplest deployment units in Kubernetes and represent a single instance of an application.

Pods consist of the following components:

Pods are ephemeral and can be created, deleted, or replaced anytime.

EKS Components: Services

Services are abstractions that define a logical set of pods and a policy to access them.

Services consist of the following components:

A service type determines how the service is exposed to the external network. There are four types of services: ClusterIP, NodePort, LoadBalancer, and ExternalName.

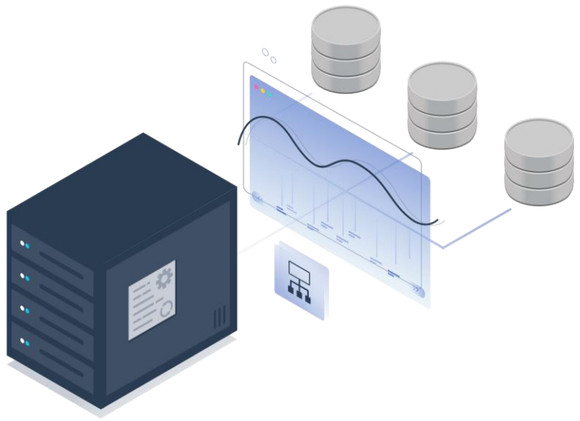
EKS Networking Modes

EKS supports two networking modes for pods: VPC-native and overlay.

- **VPC-native:** Uses AWS VPC CNI plugin, assigns VPC IP addresses to pods, and allows direct communication with AWS resources
- **Overlay:** Utilizes a third-party CNI plugin and creates a virtual network layer on VPC, enabling pods to have distinct IP addresses and communicate using encapsulation like VXLAN or GRE

EKS Load Balancing Options

Depending on the networking mode and the type of traffic, EKS supports three types of load balancers for pods:



- **Classic Load Balancer (CLB):** Layer 4; supports TCP and UDP; compatible with both modes
- **Network Load Balancer (NLB):** Layer 4; supports TCP, UDP, and TLS for VPC-native mode; preserves source IP
- **Application Load Balancer (ALB):** Layer 7; supports HTTP and HTTPS; compatible with both modes; advanced features

EKS Integration with Other AWS Services

Integrating EKS with other AWS services offers numerous advantages:

- **Streamlined operations:** Centralized management and monitoring
- **Enhanced security:** Leverage AWS security features for the clusters
- **Scalability:** Seamlessly scale resources as the application demands
- **Automation:** Leverage AWS automation tools for deployments and updates

EKS Integration with Other AWS Services

Several AWS services complement EKS:

Amazon RDS:
It provides managed databases for containerized applications.

Amazon S3:
It offers object storage for storing container artifacts and logs.

Amazon VPC:
It enables networking customization for EKS clusters.

AWS IAM:
It ensures secure identity and access management.

EKS Integration with Other AWS Services

Several AWS services complement EKS:

Amazon CloudWatch:
It supports monitoring and observability.

AWS Secrets Manager:
It allows securely managing and rotating secrets.

Amazon ECR:
It hosts Docker container image registry.

AWS App Mesh:
It facilitates service mesh for microservices communication.

EKS Integration with Other AWS Services

Best practices for EKS integration:

- Follow AWS Well-Architected Framework for design
- Implement security best practices, like IAM and VPC isolation
- Optimize costs by right-sizing resources and using AWS Cost Explorer
- Set up comprehensive monitoring and alerting with CloudWatch

EKS Integration with Other AWS Services

Best practices for EKS integration:

- Automate deployment and scaling using AWS services like CodePipeline and Auto Scaling
- Consider horizontal pod autoscaling based on metrics
- Plan for backups and disaster recovery using EBS snapshots and AWS Backup

Assisted Practice

Creating and Scaling EKS Clusters **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to implement the containerization of applications on an AWS EKS cluster by configuring and optimizing the EKS cluster for efficient and secure application deployment.

Outcome:

By performing this demo, you will learn how to implement containerization of applications on an AWS EKS cluster by configuring and optimizing the cluster for efficient and secure application deployment. This will enhance your skills in managing Kubernetes on AWS, focusing on security, scalability, and performance optimization.

Note: Refer to the demo document for detailed steps:
03_Creating_and_Scaling_EKS_Clusters

Assisted Practice: Guidelines

Steps to be followed:

1. Create the IAM role for the Amazon EKS cluster and EKS worker node group
2. Create the AWS EKS cluster
3. Create the AWS EKS worker node group

ASSISTED PRACTICE

Assisted Practice

Deploying Kubernetes YAML with CodePipeline **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to setup an automated deployment process that integrates AWS CodePipeline with Kubernetes, allowing seamless application deployment to an EKS cluster.

Outcome:

By performing this demo, you will learn how to set up an automated deployment process that integrates AWS CodePipeline with Kubernetes, allowing for seamless application deployment to an EKS cluster. This will enhance your continuous integration and deployment (CI/CD) skills with Kubernetes on AWS, ensuring efficient and reliable application delivery.

Note: Refer to the demo document for detailed steps:
04_Deploying_Kubernetes_YAML_with_CodePipeline

Assisted Practice: Guidelines

Steps to be followed:

1. Create a GitHub repository
2. Create AWS CodePipeline to deploy manifests on EKS
3. Set up build commands

ASSISTED PRACTICE

Quick Check

You are managing a Kubernetes cluster using Amazon EKS and need to ensure that the control plane is functioning correctly to handle requests and manage the cluster. What is the primary function of the control plane in Amazon EKS?

A. Managing and scaling Kubernetes worker nodes

B. Handling the API requests and managing the Kubernetes cluster

C. Executing containerized applications

D. Balancing the load across worker nodes

Advanced EKS Topics

EKS Security

Security is a crucial aspect of running any Kubernetes cluster, especially in a public cloud environment.

EKS Security Best Practices

EKS provides several features and options to help secure the cluster and its resources, such as:

Using IAM roles to control cluster access

- Users can use IAM roles for service accounts to assign fine-grained permissions to pods and applications without sharing or storing any IAM credentials.
- User can also use IAM roles to grant access to other AWS services from the cluster, such as S3, DynamoDB, or SNS.

EKS Security Best Practices

EKS provides several features and options to help secure the cluster and its resources, such as:

Implementing a firewall to protect each cluster

- Users can use the AWS Network Firewall to create a stateful firewall that inspects and filters traffic to and from the cluster.
- Users can define firewall rules based on IP addresses, domains, protocols, and ports.
- Users can also use AWS WAF to protect the cluster from web attacks like SQL injection, cross-site scripting, or DDoS.

EKS Security Best Practices

EKS provides several features and options to help secure the cluster and its resources, such as:

Using SSL/TLS to encrypt cluster traffic

- Users can use AWS Certificate Manager to create and manage SSL/TLS certificates for cluster endpoints, such as the API server, the ingress controller, or the load balancer.
- Users can also use AWS KMS to encrypt data at rest, such as secrets, volumes, or snapshots.

EKS Security Best Practices

EKS provides several features and options to help secure the cluster and its resources, such as:

Accessing cluster with a bastion host

- Users can use a bastion host to securely access cluster nodes via SSH, without exposing them to the internet.
- A bastion host is a hardened EC2 instance that is a gateway between the local machine and cluster nodes.
- Users can use AWS Systems Manager Session Manager to connect to the bastion host without opening any inbound ports.

EKS Security Best Practices

EKS provides several features and options to help secure the cluster and its resources, such as:

Connecting to clusters with VPNs

- Users can use AWS VPN to create a secure and private connection between the local network and the cluster VPC.
- This way, user can access their cluster resources as if they were on their local network without exposing them to the internet.
- Users can also use AWS Direct Connect to establish a dedicated and high-performance connection between the on-premises network and their cluster VPC.

Autoscaling and Load Balancing in EKS

Ensure the performance and availability of the cluster and applications

Autoscaling and Load Balancing in EKS

EKS supports several options for autoscaling and load balancing, such as:

Cluster Autoscaler

- The Cluster Autoscaler automatically adjusts the number of nodes in the cluster based on the demand and the capacity of node groups.
- It scales up the nodes when there are pending pods and scales down the nodes when they are underutilized.

Autoscaling and Load Balancing in EKS

EKS supports several options for autoscaling and load balancing, such as:

Horizontal Pod Autoscaler

- The Horizontal Pod Autoscaler automatically adjusts the number of pods in a deployment or a replica set based on the CPU utilization or custom metrics.
- It scales up the pods when the load increases and scales down the pods when the load decreases.

Autoscaling and Load Balancing in EKS

EKS supports several options for autoscaling and load balancing, such as:

Vertical Pod Autoscaler

- The Vertical Pod Autoscaler automatically adjusts the CPU and memory requests and limits of the pods based on the historical usage and the available resources.
- It scales up the pods when they need more resources and scales down the pods when they need fewer resources.

Autoscaling and Load Balancing in EKS

EKS supports several options for autoscaling and load balancing, such as:

AWS Load Balancer Controller

- The AWS Load Balancer Controller automatically creates and manages AWS load balancers for cluster services and ingresses.
- It supports Network Load Balancers and Application Load Balancers and provides features such as SSL/TLS termination, path-based routing, health checks, and cross-zone load balancing.

Autoscaling and Load Balancing in EKS

EKS supports several options for autoscaling and load balancing, such as:

ALB Ingress Controller

- The ALB Ingress Controller automatically creates and manages Application Load Balancers for cluster ingresses.
- It supports SSL/TLS termination, path-based routing, health checks, and cross-zone load balancing features.

Assisted Practice

Implementing EKS Security Policies and Network Policies **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to create an AWS network and security policies on an EKS cluster.

Outcome:

By performing this demo, you will learn how to create an AWS network and security policies on an EKS cluster. This will enhance your ability to manage and secure Kubernetes clusters on AWS, ensuring robust network configurations and compliance with security best practices.

Note: Refer to the demo document for detailed steps:
05_Implementing_EKS_Security_Policies_and_Network_Policies

ASSISTED PRACTICE

Assisted Practice: Guidelines

Steps to be followed:

1. Create AWS network and security policies on EKS cluster

ASSISTED PRACTICE

Assisted Practice

Autoscaling and Load Balancing with EKS **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to implement containerization of applications on an AWS EKS cluster, including autoscaling and load balancing for improved scalability and availability.

Outcome:

By performing this demo, you will learn how to implement containerization of applications on an AWS EKS cluster, including autoscaling and load balancing. This will enhance your ability to ensure improved scalability and availability of applications through efficient Kubernetes management on AWS.

Note: Refer to the demo document for detailed steps:
06_Autoscaling_and_Load_Balancing_with_EKS

Assisted Practice: Guidelines

Steps to be followed:

1. Create the AWS EKS autoscaling

ASSISTED PRACTICE

EKS Cluster Maintenance and Upgrades

These are essential tasks to keep the cluster secure and up-to-date with the latest Kubernetes features and bug fixes.

EKS Cluster Maintenance and Upgrades

EKS provides several options and tools to help maintain and upgrade the cluster, such as:

Updating cluster control plane

- Users can update the Kubernetes version of the cluster control plane using the EKS console, the AWS CLI, or the AWS SDK.
- EKS supports in-place updates of the control plane, which means that users don't need to create a new cluster or migrate the workloads.
- EKS performs the update in a rolling fashion, replacing the old control plane nodes with the new ones while maintaining the cluster's availability and functionality.

EKS Cluster Maintenance and Upgrades

EKS provides several options and tools to help you maintain and upgrade the cluster, such as:

Updating nodes in the cluster

- Users can update the Kubernetes version of the cluster nodes using the EKS console, the AWS CLI, or the AWS SDK.
- EKS supports in-place updates of the node groups, which means that users don't need to create new node groups or drain the nodes manually.
- EKS performs the update in a rolling fashion, replacing the old nodes with the new ones, while maintaining the availability and functionality of the workloads.

EKS Cluster Maintenance and Upgrades

EKS provides several options and tools to help maintain and upgrade the cluster, such as:

Updating Kubernetes add-ons and custom controllers

- Users can update the Kubernetes add-ons and custom controllers they use in their cluster, such as CoreDNS, kube-proxy, or the AWS Load Balancer Controller.
- EKS supports updating the add-ons and custom controllers using the EKS console, the AWS CLI, or the AWS SDK.
- EKS performs the update by applying the new manifests to the cluster while ensuring the compatibility and functionality of their add-ons and custom controllers.

EKS Cluster Maintenance and Upgrades

EKS provides several options and tools to help maintain and upgrade the cluster, such as:

Updating Kubernetes manifests

- Users can update the Kubernetes manifests they use to deploy and manage their applications and resources in the cluster, such as deployments, services, or ingresses.
- Users can use tools such as kubectl, Helm, or eksctl to apply the new manifests to their cluster while ensuring the compatibility and functionality of their applications and resources.

EKS Monitoring and Logging

These are essential components used for troubleshooting and optimizing the cluster and applications.

EKS Monitoring and Logging

EKS integrates with several AWS services and tools to provide monitoring and logging for the cluster, such as:

CloudWatch Container Insights

- CloudWatch Container Insights provides a comprehensive monitoring solution for EKS at the cluster, node, pod, task, and service level.
- It collects metrics, logs, and events from the cluster and displays them in pre-built dashboards and widgets.
- It also supports Prometheus metrics and alerts and provides anomaly detection and resource optimization features.

EKS Monitoring and Logging

EKS integrates with several AWS services and tools to provide monitoring and logging for the cluster, such as:

CloudWatch Logs

- CloudWatch Logs provides a solution for EKS that allows users to store, search, and analyze the logs from their cluster control plane, nodes, and pods.
- It also supports log filtering, querying, and exporting features.
- Users can use the AWS Load Balancer Controller, Fluent Bit, or Fluentd to send the container logs to CloudWatch Logs.

EKS Monitoring and Logging

EKS integrates with several AWS services and tools to provide monitoring and logging for the cluster, such as:

CloudWatch Alarms

- CloudWatch Alarms provides an alerting solution for EKS that allows the monitoring of the metrics and logs from the cluster and triggers actions when they cross a threshold.
- Users can use CloudWatch Alarms to notify them of any issues or anomalies in their cluster, such as high CPU utilization, low disk space, or failed pods.
- Users can also use CloudWatch Alarms to trigger actions such as scaling, restarting, or sending notifications.

EKS Monitoring and Logging

EKS integrates with several AWS services and tools to provide monitoring and logging for the cluster, such as:

CloudFormation

- CloudFormation offers EKS templating for consistent resource provisioning.
- It can manage the cluster, node groups, add-ons, and controllers.
- Additionally, it handles related AWS resources like VPCs, subnets, and security groups.

Assisted Practice

Creating EKS Cluster Using CloudFormation Template **Duration: 20 Min.**

Problem Statement:

You have been assigned a task to automate the provisioning of cloud resources using the CloudFormation service by creating a template for launching a new EKS Cluster.

Outcome:

By performing this demo, you will learn how to automate the provisioning of cloud resources using AWS CloudFormation by creating a template for launching a new EKS cluster. This will enhance your skills in infrastructure as code (IaC) and efficient resource management on AWS.

Note: Refer to the demo document for detailed steps:
07_Creating_EKS_Cluster_Using_CloudFormation_Template

Assisted Practice: Guidelines

Steps to be followed:

1. Create a new CloudFormation template to launch a new VM

ASSISTED PRACTICE

Quick Check

You are managing an Amazon EKS cluster and want to ensure that you are keeping track of its performance and health. What is the purpose of EKS Monitoring and Logging?

A. To automatically adjust the number of Kubernetes nodes in the cluster based on demand

B. To monitor and log the performance and health of the EKS cluster

C. To balance the load across worker nodes in the cluster

D. To create a new EKS cluster using a CloudFormation template

Key Takeaways

- Docker-based applications consume less memory, start and stop faster, and use system resources better.
- Pods are the smallest deployable computing units in Kubernetes. They can be created and managed within a Kubernetes cluster.
- A Kubernetes cluster is a group of physical or virtual machines called nodes to manage containerized apps and services.
- Amazon EKS is a managed Kubernetes service on AWS that simplifies the deployment and management of Kubernetes clusters.
- AWS Load Balancer Controller automatically creates and manages AWS load balancers for cluster services and ingresses.

Deploying a Kubernetes Pod Using EKS

Duration: **30 Minutes.**

Project Agenda: To deploy Jenkins on Amazon Elastic Kubernetes Service (EKS) as a Kubernetes Pod

Description:

This project guides you through the process of deploying Jenkins, a widely-used Continuous Integration and Continuous Deployment (CI/CD) tool, on Amazon EKS. By the end of this guide, you will have a running Jenkins instance as a Kubernetes pod on EKS, accessible via a Load Balancer.

LESSON-END PROJECT

Deploying a Kubernetes Pod Using EKS

Duration: **30 Minutes.**

Perform the following :

1. Deploy Jenkins on EKS as a Kubernetes Pod

Expected Deliverables: A running Jenkins instance on EKS accessible via a Load Balancer

LESSON-END PROJECT

Thank you!